

Universidad Tecnológica Nacional
Tecnicatura Universitaria en Programación

Nombre de la materia: Programación 2

Título del proyecto: Sistema de Pedidos - Food Store

Datos de los estudiantes:

Nombre: Nicolas

Apellido: Silvan

Numero de legajo: 102222

Comisión: 7

Nombre: Gonzalo

Apellido: Cano

Numero de legajo: 102615

Comisión: 2

Fecha de entrega: 21/06/2026

Índice

- I. [Marco Teórico](#)
- II. [Decisiones Técnicas](#)
- III. [Dificultades y Soluciones](#)
- IV. [Bibliografía y Links](#)

Marco Teórico

El sistema de pedidos desarrollado se apoya en Java 21, un lenguaje robusto y multiplataforma que permite implementar aplicaciones con orientación a objetos y manejo eficiente de colecciones.

El uso de POO se refleja en la herencia de la clase Base, el uso de interfaces como Calculable, y la aplicación de encapsulamiento y polimorfismo en las entidades (Producto, Usuario, Pedido, etc.).

Las relaciones entre clases siguen el modelo definido en el UML, donde se representan asociaciones 1: N y composiciones, como la de Pedido con DetallePedido. Para gestionar estas relaciones se emplean colecciones en Java, principalmente ArrayList, que permiten almacenar y manipular listas dinámicas de objetos.

El proyecto también incorpora el manejo de excepciones, creando clases específicas como PrecioInvalidoException, StockInvalidoException o EntidadNoEncontradaException, que garantizan la integridad de los datos y evitan inconsistencias en la lógica de negocio.

Finalmente, el uso de Git como sistema de control de versiones asegura un desarrollo ordenado y colaborativo, con historial de cambios y documentación clara.

Decisiones Técnicas

Explicación:

La solución se abordó aplicando los principios de la Programación Orientada a Objetos en Java.

Se definió una clase base (Base) que concentra atributos comunes como id, createdAt y eliminado, y a partir de ella se derivaron las entidades principales (Producto, Usuario, Pedido, Categoria).

Se utilizó una interfaz (Calculable) para abstraer el cálculo de totales en los pedidos, y se implementaron excepciones personalizadas para validar reglas de negocio, garantizando integridad en los datos.

El sistema se diseñó para funcionar en consola, con menús interactivos que permiten realizar operaciones CRUD sobre las entidades.

Organización:

El proyecto se organizó en paquetes siguiendo una arquitectura modular:

- entities/ => Contiene las clases del modelo (Producto, Usuario, Pedido, Categoria y DetallePedido).
- service/ => Implementa la lógica de negocio y validaciones (ProductoService, UsuarioService, PedidoService y CategoriaService).
- menu/ => Define los menús de consola para interacción con el usuario (ProductoMenu, UsuarioMenu, PedidoMenu y CategoriaMenu).
- exception/ => Agrupa las excepciones personalizadas (PrecioInvalidoException, StockInvalidoException, etc).
- enums/ => Contiene enumeraciones para estados y roles (Estado, FormaPago y Rol).
- Main.java => Punto de entrada del sistema, conecta los servicios y menús, y carga datos de prueba.

Capturas:

Creación de Categoría

```
--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menu principal
Seleccione:
2

--- Crear categoria ---
Nombre:
Postres
Descripcion:
Helados, tortas y postres caseros
Categoria creada con exito! ID : 3
```

Eliminar Producto

```
}
Producto {
- ID : 3
- Nombre : Hamburguesa Clasica
- Descripcion : Completa con papas
- Precio : $4500.0
- Stock : 30
- Disponible : Si
- Categoria : Comidas
}
Producto {
- ID : 4
- Nombre : Papas Fritas
- Descripcion : McCain Golden Long
- Precio : $2200.0
- Stock : 40
- Disponible : Si
- Categoria : Comidas
}

Ingrese el ID del producto a eliminar:
2
Confirma eliminar 'Agua mineral 500ml'? (S/N): s
Producto eliminado con exito.
```

Crear Pedido

```
ID del usuario:
2
Seleccione forma de pago:
1. TARJETA
2. TRANSFERENCIA
3. EFECTIVO
Opcion:
1
```

```
ID del producto a agregar:
1
Cantidad:
3
Detalle agregado. Subtotal acumulado: $4500.0
Desea agregar otro producto? (S/N): s
```

```
ID del producto a agregar:
3
Cantidad:
1
Detalle agregado. Subtotal acumulado: $13500.0
Desea agregar otro producto? (S/N): n
Pedido creado con exito. ID : 0 | Total: $13500.0
```

Listar Usuarios

```
--- USUARIOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver al menu principal
Seleccione:
1

--- Listado de usuarios ---
Usuario {
  - ID : 1
  - Nombre : Gonzalo Cano
  - Mail : gonzalo@mail.com
  - Celular : 1122334455
  - Rol : ADMIN
}
Usuario {
  - ID : 2
  - Nombre : Nicolas Silvan
  - Mail : nico@mail.com
  - Celular : 1133445566
  - Rol : USUARIO
}
```

Dificultades y Soluciones

Durante el desarrollo del sistema, el mayor desafío técnico estuvo en la implementación de la clase Pedido y su relación de composición con DetallePedido, ya que, a diferencia del resto de las entidades, que tienen un CRUD relativamente directo, Pedido necesitaba coordinar varias reglas de negocio al mismo tiempo: validar stock disponible, calcular subtotales, y mantener el total general actualizado a través de la interfaz Calculable.

El problema concreto era qué pasaba si, durante la carga de un pedido con varios productos, uno de los detalles fallaba a mitad de camino, por ejemplo, por falta de stock. Si no se manejaba con cuidado, el sistema podía terminar con un pedido a medio cargar en la colección en memoria, con algunos detalles guardados y el total mal calculado.

Para resolverlo, se decidió que el método addDetallePedido lance una excepción propia (StockInvalidoException o CantidadInvalidaException) apenas detecta el problema, y que el PedidoService capture ese error y elimine el pedido completo de la colección antes de propagar el error al menú. Esto garantiza que en memoria nunca quede un pedido inconsistente, cumpliendo con la regla de negocio definida en la consigna.

También costó encontrar el equilibrio correcto en la separación de responsabilidades entre las entidades, los servicios y los menús de consola.

Al principio era tentador resolver validaciones directamente en el menú (por ejemplo, chequear el stock ahí mismo antes de llamar al service), pero esto generaba duplicación y rompía la idea de tener una capa de servicios responsable de las reglas de negocio.

La solución fue mantener una regla clara: el menú solo lee datos del usuario y muestra mensajes, las entidades solo modelan el dominio, y toda validación o regla de negocio vive exclusivamente en los servicios, que son los que lanzan las excepciones propias cuando algo no es válido.

Bibliografía

Java 21: [JDK 21 Documentation - Home](#)

Excepciones: [Lesson: Exceptions \(The Java™ Tutorials > Essential Java Classes\)](#)

Colecciones: [Collection \(Java SE 21 & JDK 21\)](#)

UML: [About the Unified Modeling Language Specification Version 2.5.1](#)

Git: [Git - Reference](#)

Links

Repositorio: <https://github.com/NicolasSilvan/TPI-Programacion2>

Video explicativo:

https://drive.google.com/file/d/1bCH_bBV88IYaCuMa_RHMpae0q8SRxb-r/view?usp=sharing